

CS135 Project A Part 1 Report

1A : Bag-of-Words Design Decision Description

When creating our bag of words, we utilized sklearn's CountVectorizer class. By default, the fit function removes punctuation and converts all letters into lowercase. In addition, we also enabled the binary parameter, which means we track present/absent values instead of count values when transforming the text. We did this because if a complex word appears just once in a text, that should automatically raise the text's reading level. Any additional appearances would only have a marginal effect on reading level. In addition, we also used the TfidfTransformer class, which assigns weightings based on the frequency that vocabulary words appear in the texts. Our approach does not consider words in the test that are not part of our vocabulary.

The final vocabulary set was determined by searching over combinations of min_df and max_df and performing 10-fold cross-validation. The best configuration was setting min_df = 5 and max_df = 1.0, which means that our vocabulary includes words that appear in at least 5 documents. Our final vocabulary size was 6456 words and had an AUCROC of 0.8029.

1B : Cross Validation Design Description

We measured AUCROC during the cross-validation process. For each combination of min_df and max_df, we found the AUC by taking the average across 10 folds. Since we used 10 folds and our training set has 5557 entries, each fold contains approximately 556 entries, and there are 9 training folds with the last one being the validation fold. We saved the combination of min_df and max_df that resulted in the highest mean AUCROC across 10 folds. After using sklearn's count vectorizer to measure how often words appear in a text, we used sklearn's TfidfTransformer to give these counts weights.

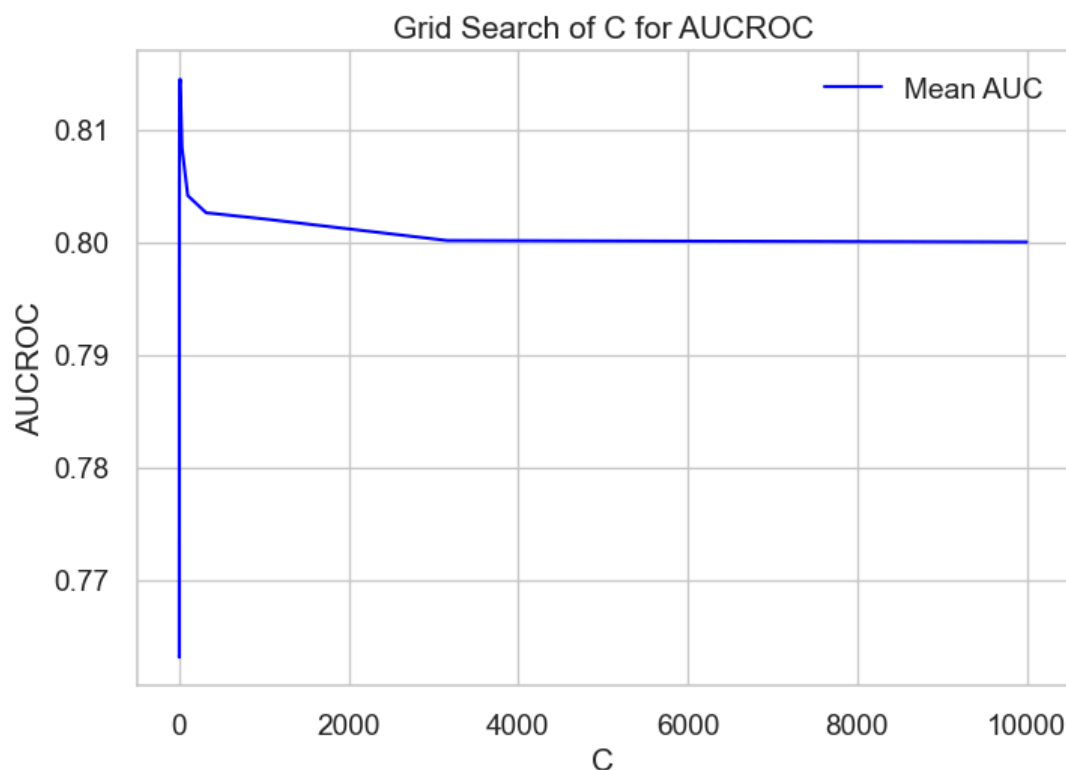
1C : Hyperparameter Selection for Logistic Regression Classifier

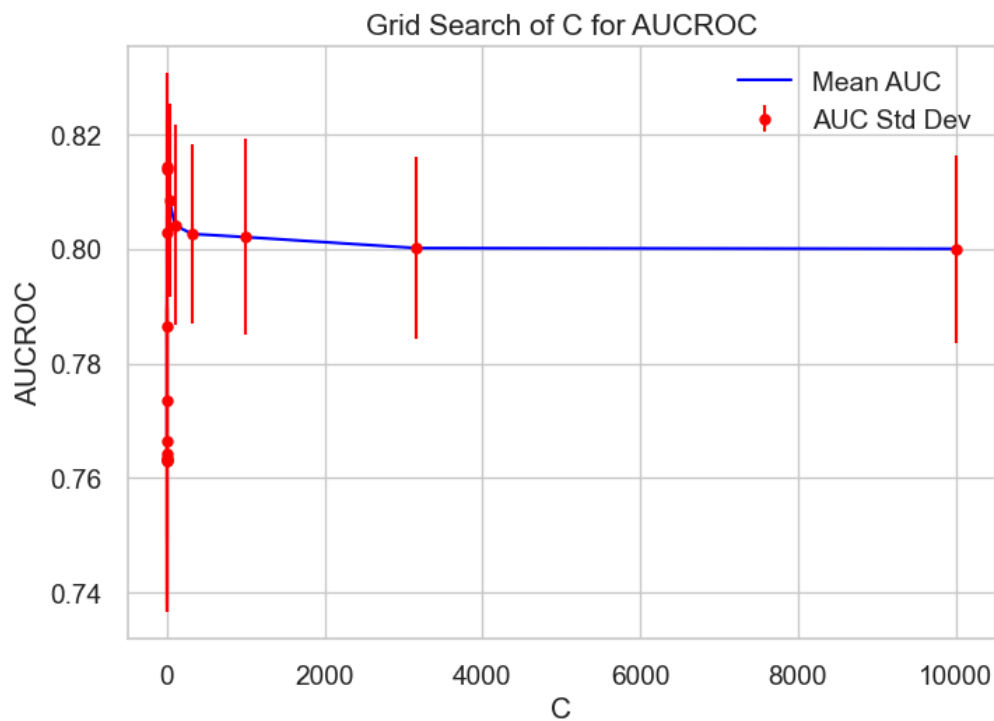
Our chosen classifier was based on a bag of words. This is ideal for this situation because it examines word frequencies and can handle high dimensional data matrices. Most of the given passages are relatively short and will only use a small percentage of the total words in the bag of words, so the bag of words was ideal for handling sparse matrices.

We first searched over the vocab size with a range from 5 to 50 (inclusive and increasing by 5) for min_df and a range from (0.4, 1.0) for max_df with each max_df increasing by 0.1. Increasing the min_df and lowering the max_df means that we modified the bag of words to only count words with a meaningful frequency. The potential advantage of changing the min_df and

max_df in this way is that our model will not train on words that do not appear very often in the training data. If a word only appears two times in the training text, that word should not be counted because it can possibly lead to overfitting, where any passage with that word would be easily classified as low or high level. Lowering the min_df and increasing the max_df increases our model's complexity, but at the risk of overfitting to noise that may appear. We chose these values for min_df because they would filter out words seldom used in the 5557 passages.

Next, we chose to search for the best regularization parameter (C). By adding weak or strong regularization, we could choose how general or complex the model would be. A high C reduces regularization and allows for more complex models, while a low C increases regularization and makes the model more generalizable. We then searched over C with a grid of logspace(-4, 4, 17). However, we found that the optimized C value of 10.0 with AUCROC of 0.8145 for our training set led to overfitting on the development set. The difference between the training and test AUCROC with C = 10.0 was much larger than the difference when C = 1.0. As such, because we had to submit each time to view the test set AUCROC, we settled on a C value of 1.0 to reach the threshold.





AUC of: 0.763247893438173 for C of: 0.0001

AUC of: 0.7631245124539234 for C of: 0.00031622776601683794

AUC of: 0.7631533158778683 for C of: 0.001

AUC of: 0.7634194141056869 for C of: 0.0031622776601683794

AUC of: 0.7642576298811126 for C of: 0.01

AUC of: 0.766408578888915 for C of: 0.03162277660168379

AUC of: 0.7736012398252206 for C of: 0.1

AUC of: 0.7864533970233752 for C of: 0.31622776601683794

AUC of: 0.8028884734091463 for C of: 1.0

AUC of: 0.8140312774270761 for C of: 3.1622776601683795

AUC of: 0.8145228968337355 for C of: 10.0

AUC of: 0.8084695222371876 for C of: 31.622776601683793

AUC of: 0.8041796601429525 for C of: 100.0

AUC of: 0.8026510160901921 for C of: 316.22776601683796

AUC of: 0.8021003070313067 for C of: 1000.0

AUC of: 0.8001804480641648 for C of: 3162.2776601683795

AUC of: 0.8000500430972963 for C of: 10000.0

Note: error is calculated using np.std of the list of AUCs across 10 folds.

From the graph, we see that AUCROC steadily increases from $C = 0.0001$ to $C = 10.0$ declines sharply from $C = 10.0$ to $C = 100.0$, and finally plateaus from $C = 100.0$ onward. These results could indicate overfitting, as optimal C values typically lie in the range of $[10^{-4}, 10^{-3}]$. An optimal $C = 10.0$ means that the feature weights we derived were quite high and thus required a high degree of regularization.

1D : Reflecting on Your Test Set Performance

Our test set AUCROC was 0.69716, compared to a validation AUCROC of 0.8029. The significant decline in performance is likely due to the fact that the vocabularies of the training and test data are very different. Because validation was done on a 1/10th fold of the training data, the vocabulary we used in training still generalized quite well on validation data. However, test data is independent of training data, so if the vocabulary of the test data is very different from that of the training data, then it makes sense that our model generalized worse on test data.